

MODUL PRAKTIKUM SISTEM EMBEDDED

Mendalami Komunikasi Serial UART

Analisis Arsitektur, Protokol Data, dan Strategi Optimalisasi pada STM32 & ESP32

PENYUSUN

Abby Putro Nugroho

40040324620017

PROGRAM STUDI

Teknologi Rekayasa Otomasi

Sekolah Vokasi - UNDIP

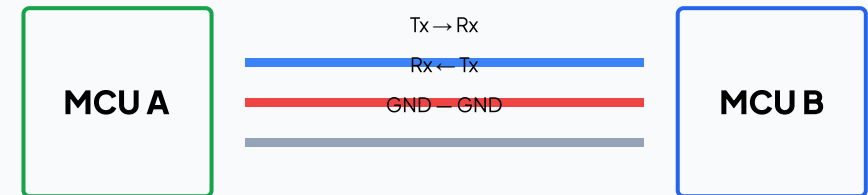
01. APA ITU UART?

UART adalah peripheral hardware yang mengubah data antara format ****paralel**** (internal MCU) dan ****serial**** (jalur komunikasi).

- **Asinkron:** Tidak membutuhkan jalur clock bersama. Receiver melakukan sampling berdasarkan deteksi Start Bit.
- **Full Duplex:** Jalur kirim (Tx) dan terima (Rx) bersifat independen, memungkinkan pengiriman dua arah simultan.
- **Peer-to-Peer:** Didesain hanya untuk komunikasi antara dua perangkat saja.

Penting: Karena tidak ada clock, kedua perangkat harus memiliki *Baud Rate* yang identik. Perbedaan >2% akan merusak integritas data.

DIAGRAM KONEKSI FISIK



02. BEDAH FRAME DATA UART

Satu karakter data dibungkus dalam paket (Frame) dengan struktur:

Start Bit (1-bit): Logika Low (0). Menandai akhir kondisi Idle dan membangunkan receiver.

Data Bits (8-bit): Karakter utama, dikirim mulai dari bit paling rendah (LSB).

Parity Bit (Optional): Bit cek error (Even/Odd). Jika jumlah bit '1' tidak sesuai, data dianggap korup.

Stop Bit (1/2-bit): Logika High (1). Jeda untuk reset kondisi garis ke Idle.

Konfigurasi 8N1

8 Data bits, No parity, 1 Stop bit. Ini adalah standar universal yang paling stabil untuk komunikasi jarak dekat hingga menengah.

Apa itu Baud Rate?

Jumlah simbol (bit) yang dikirim per detik. Standar: 9600 (industri), 115200 (debug PC). Kecepatan tinggi membutuhkan kabel yang lebih pendek dan berkualitas tinggi.

03. MEKANISME SAMPLING & ERROR

Teknik 16x Oversampling

Hardware UART tidak hanya membaca satu kali per bit. Ia melakukan sampling 16 kali lebih cepat dari baud rate untuk menemukan titik tengah yang paling stabil. Ini meminimalisir kesalahan akibat *noise* listrik.

```
// Majority Voting Logic
Sample_7, Sample_8, Sample_9 == 1?
Result = BIT_HIGH;
```

Jenis Error Umum:

Framing Error: Stop bit tidak ditemukan pada waktunya (Baud rate tidak cocok).

Overrun Error: Data baru menimpa data lama sebelum sempat dibaca oleh CPU.

Noise Error: Sinyal terganggu di tengah transmisi.

04. POLLING, INTERRUPT, & DMA

Metode	Cara Kerja	Beban CPU	Kegunaan
Polling	CPU terus-menerus mengecek status register Rx secara manual (Blocking).	100% (Sangat Berat)	Testing dasar atau komunikasi sederhana satu arah.
Interrupt	CPU dipanggil oleh hardware hanya saat ada data masuk (Event Driven).	Medium (Tergantung trafik)	Menerima perintah singkat dari sensor atau user input.
DMA	Data pindah otomatis dari hardware ke RAM tanpa melibatkan CPU sama sekali.	0% (Sangat Ringan)	Transfer data besar (MB), update firmware, atau stream GPS.

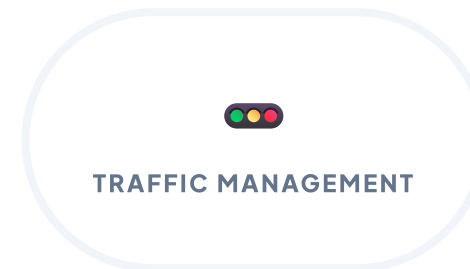
05. HARDWARE FLOW CONTROL

Masalah: Pengirim mengirim data lebih cepat daripada kemampuan penerima memprosesnya (Buffer Full).

Solusi: RTS/CTS

CTS (Clear to Send): Transmitter mengecek pin ini; jika LOW, ia boleh mengirim.

RTS (Request to Send): Receiver menarik pin ini ke LOW jika buffernya masih muat.



06. ESP32 VS STM32 PERIPHERAL

ESP32

Remappable Pin: Bisa menggunakan hampir semua pin GPIO untuk Tx/Rx via IO MUX.

3 Port UART: Terdiri dari UART0 (Debug), UART1, dan UART2.

FIFO Buffer: Kapasitas 128 byte per port, sangat membantu saat OS (FreeRTOS) sedang sibuk.

STM32

Fixed Alternate Function: Pin Tx/Rx sudah ditentukan di pabrik (lebih kaku tapi latensi rendah).

USART Support: Bisa mode Sinkron (butuh pin clock) selain Asinkron.

DMA Matang: Integrasi DMA yang sangat fleksibel untuk aplikasi real-time industri.

07. INTEGRITAS DATA & RING BUFFER

Cyclic Redundancy Check (CRC)

Menambahkan kode matematis di akhir paket data. Jika hasil hitung di receiver berbeda, data dibuang. Ini adalah standar wajib di lingkungan industri dengan noise motor listrik tinggi.

CRC Tip: Gunakan hardware CRC engine jika tersedia di MCU untuk performa maksimal.

Konsep Ring Buffer

Buffer melingkar yang memisahkan antara proses ****Penerimaan**** (oleh Interrupt/DMA) dan proses ****Pengolahan**** (oleh Main Loop). Menjamin tidak ada data yang hilang saat CPU sedang memproses algoritma berat.



08. IDLE LINE & TIMEOUT PARSING

Bagaimana cara tahu sebuah pesan sudah selesai jika panjangnya tidak tetap (*variable length*)?

Idle Line Detection (IDLE)

Fitur pada STM32 yang memicu interupsi ketika jalur Rx diam selama durasi 1 frame data. Ini menandakan pengiriman paket data utuh telah selesai.

```
// Logika Timeout Sederhana
if (millis() - last_received > TIMEOUT_MS) {
    process_full_packet();
    reset_buffer();
}
```

KESIMPULAN STRATEGIS

"Memahami UART bukan sekadar tentang menghubungkan Tx dan Rx, tapi tentang mengelola sumber daya CPU dan menjaga integritas data."

Best Practice:

Gunakan DMA untuk transfer data di atas 115200 bps.
Selalu gunakan Ring Buffer untuk memisahkan konteks ISR dan aplikasi utama agar sistem tidak hang.

Pilihan Hardware:

Pilih ESP32 untuk kemudahan layout PCB. Pilih STM32 untuk kebutuhan protokol industri yang lebih presisi dan variatif.

Terima Kasih

Abby Putro Nugroho | SV UNDIP '24